Prism Capital

Institutional-grade Shariah-compliant private-placement platform.

User guide + technical reference · Formerly RibhShare

Version: 2026-07-24 · Phases P1–P4 shipped
Contents: product overview · roles · end-to-end flow · screen reference · profit maker-checker · ledger · schema · RPCs · deploy

Prism Capital — User Guide

Formerly RibhShare. Institutional-grade Shariah-compliant private-placement platform.

1 · Product overview

Prism Capital lets a **Line Manager** raise Shariah-compliant capital from **Investors** for real projects, with **CEO** oversight. Every rupee is tracked in **whole units** (not free-form amounts), every profit distribution passes a **maker–checker** gate, and every posting hits an **immutable double-entry ledger**.

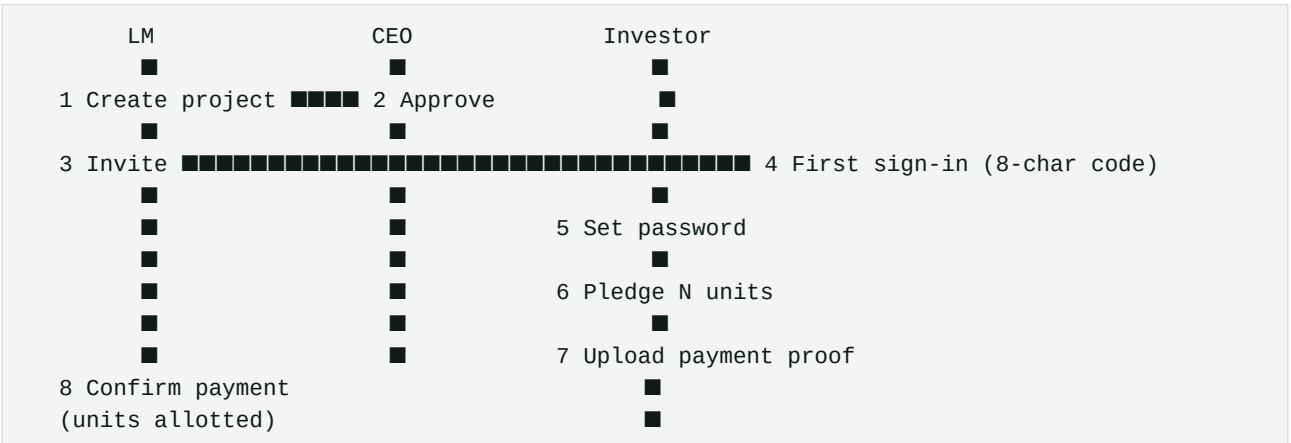
- **Currency**: Naira (₦), stored as minor units (kobo) internally.
- **Web app**: Expo Web SPA on Netlify → `ribhshare.com`.
- **Backend**: Supabase (Postgres + Auth + Edge Functions + Row-Level Security).

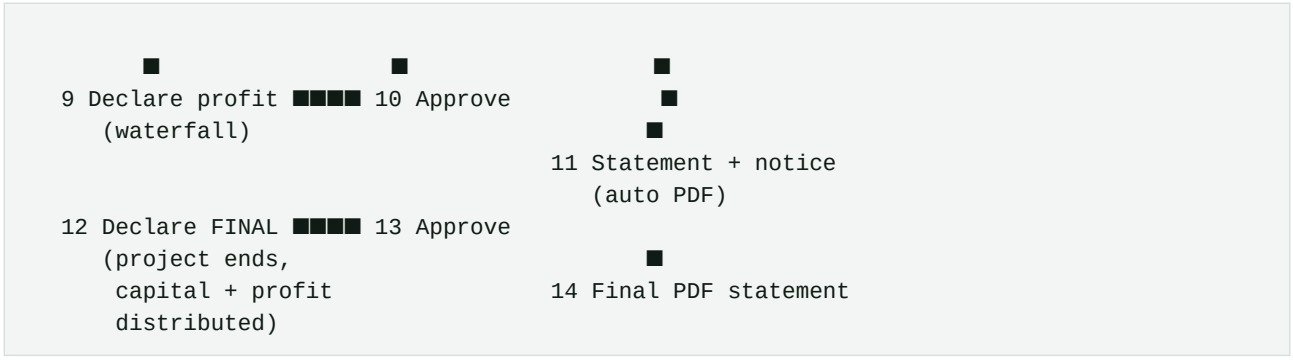
2 · Roles

Role	What they do
CEO / Admin	Approves projects, approves profit declarations (four-eyes), sees platform-wide trial balance, exports ledger CSV.
Line Manager (LM)	Creates projects, invites investors, confirms bank payments, declares profits, ends projects.
Investor	Receives invite → sets password → pledges units → uploads payment proof → receives distribution notices + PDF statements.

Test accounts (dev): see `/app/memory/test_credentials.md`.

3 · End-to-end lifecycle





4 · Screen-by-screen — Line Manager

4.1 Home

- Hero: total AUM, active projects, pending approvals.
- **Manager Earnings** card (green): total manager share across all projects.
- **Profit Sources** list: every project with realised profit, sorted by LM cut.

4.2 Projects tab

- Grid of your projects. Each card: code, name, stage chip (INITIATION / ACCEPTANCE / PROGRESS / END), target vs raised.
- **"+ New Project"** button opens the wizard.

4.3 Create Project wizard (5 steps)

1. **Basics** — Name, description, target ■, **Total Units**, **Min units/investor**, **Prism fee %** (default 7.5%), duration.

2. **Financials** — Manager profit share % (0–50%, default 30%).

3. **Documents** — 3 required uploads (Feasibility, Contract, ID) + optional banner.

4. **Review**.

5. **Submit** → CEO approval queue.

4.4 Project Detail (tabs)

- **Overview** — Progress bar, unit spectrum (subscribed vs available), key facts.
- **Documents** — All uploaded files.
- **Investors** — Table with `units · ■ · payment reference (PRSM-CODE-INV###)`. "Copy invite link" pill on unredeemed invites.
- **Activity** — Announcements + fund-use posts.
- **Profits** ★ — Waterfall calculator + declaration form (see §5).
- **Ledger** ★ — Every DR/CR posted to this project. Balances at top.
- **Audit** ★ — Colour-coded event stream. CSV export button.

- **Reconciliation ★** — Expected inflow vs claimed per invite. Variance flagged red.

4.5 Earnings tab

- LM-only. Cross-project view of manager share, per-project breakdown, timeline of recent profit posts.

5 · Profit declaration flow (maker–checker)

LM (maker) submits a declaration on the Profits tab:

- Enters `gross` and `costs` in ■.
- Live waterfall preview updates:

```
Gross          ■1,000,000
– Costs        ■  200,000
= Net          ■  800,000
– Prism fee    ■   60,000   (7.5% of net)
= Distributable ■  740,000
■■ Manager     ■  222,000   (30% of distributable)
■■ Investor pool ■ 518,000   (70%)
   Per unit    ■   10,360   (÷ 50 units)
```

- Two buttons:

- **Submit for approval** — regular distribution, project stays in PROGRESS.

- **Submit as final (end project)** — final distribution + capital return; flips stage to END on approval.

CEO (checker) sees it in Approvals → Declarations queue:

- Cannot approve their own submissions (four-eyes principle).
- On approve: waterfall is locked into `profit_declarations` as immutable record; one `distribution_notice` per confirmed investor is minted; ledger auto-posts.
- On reject: adds a `rejection_note`; declaration is preserved for audit.

6 · Screen-by-screen — Investor

6.1 First-time sign-in

- Investor receives email with 8-char code (or copy-invite-link from LM if email undelivered).
- `/first-signin` → email + code → magic-link validated → `/set-password` → dashboard.

6.2 Portfolio

- Aggregate stats: invested, projected profit, realised share.
- Cards per holding: project, units held, ownership %, per-holding realised profit.

6.3 Project Detail (investor view)

- **Overview** — same as LM but read-only.
- **Payment** — "How many units?" input (integer, ≥ min per investor), live pledge total, min-units hint. After pledge: big monospaced `PRSM-CODE-INV###` reference to copy into transfer narration.
- **Financials** — Project target, your capital, projected profit, realised share, ownership %.
- **Activity** — Announcements and fund-use updates.

6.4 Statements tab ★

- Every `distribution_notice` renders as a card:

```
PRSM-ABC-NOT001 · 2026-07-24
Regular distribution

Gross           ■1,000,000
Costs         −■  200,000
Net           ■  800,000
Prism fee     −■   60,000
Pool          ■  518,000
You (10 units)■  103,600
```

- FINAL notices show a highlighted "Capital returned" block with the total credit.
- **Download PDF** button per notice → generates `Statement_PRSM-<ref>.pdf` (A4, Prism-branded, immutable footer).

7 · Screen-by-screen — CEO

7.1 Dashboard

- Platform-wide KPIs: total AUM, active projects, pending approvals.
- **Trial Balance CSV ★** — top-right button. Exports current account balances across all projects (bank, investor capital, payables, realised P&L, fees, reserves).

7.2 Approvals tab

- Top toggle: **Declarations | Projects (n)**.
- Declarations: pending profit declarations queue — reference, label, gross, net, investor pool, per-unit summary. Tap → jumps into project's Profits tab.
- Projects: pending project submissions from LMs.

7.3 All project tabs

- Same as LM view, plus: **Reconciliation** and **Ledger** visible for oversight across all projects.

8 · Ledger (double-entry, append-only)

Every material action posts balanced DR/CR entries automatically. Statement-level trigger `enforce_ledger_balance` rejects any imbalanced transaction. Accounts used:

Code	Account	Direction it grows
`project_bank`	Cash raised, held by Prism	DR
`investor_capital`	Capital committed by investor	CR
`investor_payable`	Profit owed to investor	CR
`manager_payable`	Manager cut owed	CR
`platform_fee_payable`	Prism fee owed	CR
`project_realised_pnl`	Net profit recognised	DR
`rounding_reserve`	Per-unit floor residue	CR

Auto-posting triggers:

- Invite → CONFIRMED → `DR project_bank / CR investor_capital[party]`
- Declaration → APPROVED → full waterfall entries
- FINAL declaration approved → additionally `DR investor_capital[party] / CR project_bank` (capital return)

Historical data (pre-trigger deployment) is NOT auto-backfilled. Ask us if you want a one-off backfill script.

9 · Distribution notices & audit trail

- **`distribution_notices`** — one immutable row per (approved declaration × confirmed investor). Investor sees their own; LM/CEO see all on their projects.
- **`audit_events`** — append-only log of everything: project created / stage-changed / invite created / accepted / pledged / verified / declined; declaration declared / approved / rejected.
- **`project_reconciliation(project)`** RPC — expected inflow vs claimed amount per invite, flags variance for Finance. Visible in Reconciliation tab.

10 · Deliverability & communications

- Invitation emails sent via Resend (from `noreply@paynexhq.com`).
- Gmail cold-domain deliverability to `+tag` aliases can be flaky — LM always sees the 8-char code inline in the UI as a fallback (with "Copy invite link" pill).

11 · Test credentials

Role	Email	Password
CEO	`Ceo@ribhshare.com`	`Test@123`
Line Manager	`linemanager@ribhshare.com`	`Test@123`
Investor	`investor@ribhshare.com`	`Test@123`

Last updated: 2026-07-24. Living document — updated on every release.

Prism Capital — Technical Reference

1 · Stack

Layer	Tech
Frontend	Expo SDK 54, React 19, expo-router (file-based), TypeScript
State	Zustand (auth/ui), TanStack Query (server state, 15s polling on profit queries)
Styling	Design tokens (`src/theme/`), Tailwind-grade primary `#166534`, dark mode
Backend	Supabase (Postgres 15 + Auth + Storage + Edge Functions)
Email	Resend
PDF	jsPDF (client-side statement generation)
Deploy	Netlify (frontend) + Supabase managed (backend)

2 · Environment

`/app/.env`:

```
EXPO_PUBLIC_SUPABASE_URL=https://jbwerfqgqavaxdyjxfra.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=…
```

Supabase edge-function secrets (set in Dashboard):

- `RESEND_API_KEY` — Resend API key
- `SENDER_EMAIL` — verified sender (e.g. `noreply@paynexhq.com`)
- `APP_URL` — production URL for magic-link redirect

3 · Data model

3.1 `projects`

Column	Type	Note
id	uuid PK	

Column	Type	Note
code	text	`PRJ-###` sequential
name, description	text	
target_minor	bigint	in kobo
raised_minor	bigint	trigger-maintained
realised_profit_minor	bigint	trigger-maintained
stage	text	`INITIATION` / `ACCEPTANCE` / `PROGRESS` / `END`
owner_id	uuid → profiles	LM
total_units ★	int	P1
min_units_per_investor ★	int	P1
platform_fee_bps ★	int	P1, default 750 (7.5%)
profit_split_investor_bps	int	default 7000 (70%)
pledge_expiry_hours ★	int	P1, default 72

3.2 `invites` (with P1 extensions)

Column	Type	Note
id	uuid PK	
project_id	uuid → projects	
investor_id	uuid → profiles	
status	text	`INVITED` → `ACCEPTED` → `COMMITTED` → `PROOF_SUBMITTED` → `CONFIRMED` / `DECLINED`
amount_minor	bigint	derived: units × unit_price
units_pledged ★	int	
units_allotted ★	int	set on confirm
payment_reference ★	text unique	`PRSM-<code>-INV###`

Column	Type	Note
pledged_at, **pledge_expires_at** ★	timestamptz	72h expiry
verified_at, **verified_by** ★		trigger-set on confirm
payment_claim_amount_minor / _bank / _date / _narration ★		investor-declared
first_signin_code	text	8-char alnum
first_signin_code_redeemed_at	timestamptz	

3.3 `profit_declarations` ★ P2

```
id, project_id, reference (PRSM-CODE-DECL###), label, is_final,
gross_minor, costs_minor, net_minor,
prism_fee_minor, distributable_minor, investor_pool_minor,
manager_share_minor, per_unit_minor,
status (PENDING/APPROVED/REJECTED),
declared_by, declared_at,
approved_by, approved_at,
rejected_by, rejected_at, rejection_note
```

- RLS: LM/CEO on project + confirmed investors can read.
- Immutable post-approval.

3.4 `distribution_notices` ★ P3

```
id, declaration_id, invite_id, investor_id, project_id,
reference (PRSM-CODE-NOT###),
units_held, per_unit_minor, profit_minor,
capital_returned_minor,  -- > 0 only for FINAL
is_final,
created_at
```

- One row per (approved declaration × confirmed investor). Immutable.

3.5 `audit_events` ★ P3

```
id, project_id, actor_id, entity_type, entity_id,
event_type, payload jsonb, created_at
```

Trigger sources: `projects`, `invites`, `profit_declarations`.

3.6 `ledger_entries` ★ P4

```
id, transaction_ref, sequence,
project_id, account_code, party_id,
direction (DR/CR), amount_minor,
actor_id, ref_type, ref_id, memo, created_at
```

- Append-only. Statement-level trigger `enforce_ledger_balance` rejects any transaction where `sum(DR) ≠ sum(CR)` grouped by `transaction_ref`.

- Party-scoped RLS: investor sees own party rows; LM sees own projects; CEO sees all.

4 · Key RPCs

P1 — units

- `pledge_units(invite_id uuid, units int)` — investor commits N units, sets `payment_reference` and 72h expiry.
- `expire_stale_pledges(project_id uuid)` — releases pledges past `pledge_expires_at`. Called on project-detail load.
- `project_units_committed(project_id uuid) → int` — helper for spectrum bar.

P2 — profit declarations (maker–checker)

- `declare_profit(project, gross_minor, costs_minor, label, is_final) → declaration`
- `approve_profit_declaration(id) → declaration` — CEO only. Rejects self-approval. Mints notices + posts ledger. Bumps `realised_profit_minor`. If final: creates `investor_payouts` + sets stage=END.
- `reject_profit_declaration(id, note) → declaration`
- `list_pending_declarations() → declarations[]`

P3 — transparency

- `list_investor_notices() → notices[]` — auth user's own notices with full declaration context.
- `list_project_audit(project, limit) → events[]`
- `project_reconciliation(project) → { summary, invites[] }` — expected vs claimed variance.

P4 — ledger

- `post_ledger(ref, project, ref_type, ref_id, actor, lines jsonb) → void` — helper called by triggers.
- `list_project_ledger(project, limit) → { balances[], entries[] }`
- View `ledger_project_balances` — trial-balance rollup per (project × account_code).

Legacy (still active, no longer surfaced in UI)

- `post_profit_update`, `end_project_now`, `get_manager_profit_summary`, `get_investor_profit_summary`.

5 · Edge functions (`supabase/functions/`)

Function	JWT	Purpose
`create-project`	ON	Validates + creates project row + first `PROJECT_UPDATE` event. Enforces `target divisible by total_units`.
`send-invitation`	ON	LM/ADMIN only. Creates invite row, generates 8-char code, sends email via Resend or returns code inline.
`redeem-invite-code`	OFF	Public. Rate-limited (8/email + 30/IP per 10min). Returns magiclink tokenHash.

6 · Auto-posting ledger triggers (P4)

`ledger_on_invite_confirm`

Fires on `UPDATE invites SET status='CONFIRMED'`:

```
DR project_bank                      amount_minor
CR investor_capital [party=investor]  amount_minor
```

`ledger_on_declaration_approve`

Fires on `UPDATE profit_declarations SET status='APPROVED'`:

```
DR project_realised_pnl              net_minor
CR platform_fee_payable              prism_fee_minor
CR manager_payable [party=LM]        manager_share_minor
CR investor_payable [party=inv_N]    units_N × per_unit_minor  (per investor)
CR rounding_reserve                  residue
```

If `is_final=true`, additionally:

```
DR investor_capital [party=inv_N]    invite.amount_minor       (per investor)
CR project_bank                      Σ amounts
```

Both triggers are idempotent — re-firing on the same event is a no-op.

7 · SPA routing (expo-router)

Route	Screen
`/(auth)/sign-in`	Email + password login

Route	Screen
`/(auth)/first-signin`	Invite code redemption
`/(auth)/set-password`	Password setup + role-aware redirect
`/(tabs)/home`	LM home
`/(tabs)/dashboard`	CEO dashboard
`/(tabs)/portfolio`	Investor portfolio
`/(tabs)/projects`	LM projects grid
`/(tabs)/projects/create`	5-step wizard
`/(tabs)/projects/[id]`	Project detail (LM/CEO view)
`/(tabs)/portfolio/projects/[id]`	Project detail (investor view)
`/(tabs)/approvals`	CEO approvals (declarations + projects)
`/(tabs)/statements`	Investor statements + PDF download
`/(tabs)/earnings`	LM earnings across projects
`/(tabs)/invitations/[id]`	Investor invite review
`/(tabs)/profile`	Theme + account

8 · SPA deploy — Netlify

Build:

```
yarn build:web    →    /app/dist/
```

`_redirects` in `dist/`:

```
/*    /index.html   200
```

Env vars needed in Netlify → Environment:

- `EXPO_PUBLIC_SUPABASE_URL`
- `EXPO_PUBLIC_SUPABASE_ANON_KEY`

Build cmd: `yarn build:web`

Publish dir: `dist`

9 · Repository layout

```
/app
■■■ app/                       # expo-router routes
■■■ src/
■   ■■■ screens/               # Screen components by role
■   ■■■ components/projects/   # Project-detail tabs (Ledger, Audit, Profits, Reconciliatio
n, Spectrum)
■   ■■■ services/              # Supabase clients (RPC wrappers)
■   ■■■ hooks/                 # TanStack Query hooks by domain
■   ■■■ stores/                # Zustand (useAuthStore, useUiStore)
■   ■■■ theme/                 # Design tokens
■   ■■■ utils/                 # pdfStatement.ts, currency, etc.
■■■ supabase/
■   ■■■ functions/             # Deno edge functions
■   ■■■ migrations/            # Applied in order 202506… → 20260128000000_p4_ledger.sql
■■■ docs/                      # User + technical docs (this folder)
■■■ memory/PRD.md              # Living product-requirements doc
```

Last updated: 2026-07-24.